

ВАЛИДАЦИИ – допълнения

За да не се спира програмата при некоректни данни със съобщението за грешка, което сме задали в set, трябва в Main() да вложим конструкция try-catch, която да прихване грешката (и продължи програмата след нея):

```
internal class Program
{
    0 references
    static void Main(string[] args)
    {
        try
        {
            #region тук пишем кода
        }
        catch (Exception ex)
        {
            Console.WriteLine("Грешка: " + ex.Message);
        }
        Console.ReadKey();
    }
}
```

В този случай, вместо директно в Main(), вече ще пишем в частта try. Try-catch означава опитай-хвани, т.е. ще опитаме даден код и ако има нещо неправилно, то ще бъде прихванато като изключение. Също така, вместо във всяко съобщение за грешка да пишем „Грешка: “, това може да се случи на едно място в св и след това да се допише само специфичното съобщение за конкретната грешка. Така се избягва повторемост DRY. Ако решим да коригираме – правим го само от едно място. Така се изписва съобщение, но не се спира цялата програма, а тя продължава след това – в случая за по-интересно съм оставила `Console.ReadKey();`; но може да продължи с всякакъв код. `#region-#endregion` показва, че съм свила кода, който съм написала. Той е много характерен за Main(), т.к. в него се избягват блокови конструкции за свиване.

Нека си направим клас за Река, като подобрим практиките си от предходните часове.

```
class River
{
    public string Name { get; set; }
}
```

- Нека разположим нашия class River (ед. ч.) **ПОД** class Program за по-добра структура (припомням, че Main() е като съдържание на книга, с възможно най-малко блокови конструкции {}, а всичко се измества в методи или класове извън него, като в Main() само извикваме имената им.
- Решаваме какви свойства ще са ни необходимо и ги създаваме автоматично с `prop + Tab + Tab` – те си имат вградена private защита

- Създаваме си конструктор с **ctor + Tab + Tab** : той ще свързва свойствата от класа с променливите, които ще подаваме на обектите в Main()
- Нека вместо както досега PrintInfo(), методът за извеждане на екрана наречем DisplayInfo(), т.к. е по-коректен по отношение на конзолата.

```

internal class Program
{
    1 0 references
    static void Main(string[] args) {...}
}

5 references
class River
{
    1 reference      prop + Tab + Tab
    public string Country { get; set; }
    0 references
    public string Name { get; set; }
    0 references
    public int Length { get; set; }
    0 references
    public string Description { get; set; }
    0 references
    public bool IsFloatable { get; set; }
    2 references      ctor + Tab + Tab
    public River()
    {
    }
    2 references
    public void DisplayInfo()
    {
        Console.WriteLine();
    }
}

```



Public property

Private property
with public getter
and setter

На мемето се има предвид, че дори обикновени public properties без get и set (**public string Name;** **public int Length;**) работят, но са като едно гърне с мед, в което всеки може да бърка, взема, променя... с мръсни ръце. Щом няма get и set работи, но е общодостъпно и това не е добре.

Най-лесно е да сложим автоматични свойства с prop + Tab + Tab, които ще работят с вложена защита, но ако искаме да сложим и валидации, трябва от тук насетне да допълним към стария синтаксис, т.е. да се качим над public свойствата и за всяко от тях, което искаме валидация, напишем private, като спазваме, че то е със същото име, но малка буква.

```

class River
{
    private readonly string country = "България"; // защитен и невидим в Main()
    private string name;
    private int length;
    private string description;
}

```

```

private readonly string country = "България"; // защитен и невидим в Main()
private string name;
private int length;
private string description;

```

В случая, за булевата променлива решаваме, че няма нужда от нищо допълнително и тя си остава с автоматичния { get; set; }. Добавяме и на първо място една променлива country, която ще бъде

защитена с readonly атрибут, за да не се променя и дори ѝ посяваме стойността = "България". Това е единият начин да защитим като константа данни, които няма да се променят.

- **Най-късата река в света** се счита, че е **Репруа** в Абхазия, която е дълга само **18 метра**. Друг претендент е **река Роу** в Монтана, САЩ, която е около **60 метра**.
- В **България** има множество реки, които са с дължина под **50 км**. Например, река Камчия е с дължина 46 км (но с притоците си достига 245 км). Други примери за по-малки реки, посочени в официални списъци, са Ибър (19 км) и Искрецка река (22 км).

```
public string Name
{
    get => name; // или get { return name; } по-нов (кратък) или стар синтаксис
    set
    {
        if (string.IsNullOrEmpty(value))
            throw new ArgumentException("Въведете име!");
        name = value;
    }
}

2 references
public int Length
{
    get { return length; } по-стар
    set { if (value < 19)
        throw new ArgumentException("Пог 19 км е помок!");
        length = value; }
}

1 reference
public string Description
{
    get => description;
    set
    {
        if (string.IsNullOrEmpty(value) || value.Split().Length < 3)
            throw new ArgumentException("Описанието трябва да съдържа поне 3 думи!");
        description = value;
    }
}

2 references
public bool IsFloatable { get; set; } не е необходима допълнителна валидация
```

```
public string Name
{
    get => name; // или get { return name; }
    set
    {
        if (string.IsNullOrEmpty(value))
            throw new ArgumentException("Въведете име!");
        name = value;
    }
}

public int Length
{
    get { return length; }
}
```

```

        set { if (value < 19)
                throw new ArgumentException("Под 19 км е поток!");
            length = value; }
    }
    public string Description
    {
        get => description;
        set
        {
            if (string.IsNullOrEmpty(value) || value.Split().Length < 3) // 2
интервала
                throw new ArgumentException("Описанието трябва да съдържа поне 3
думи!");
            description = value;
        }
    }
    public bool IsFloatable { get; set; }

```

Правим валидациите, независимо дали по по-стария начин с `get { return length; }` или директно с по-новия `{ get => length; }`, който е и по-кратък и лесен.

След това в `set` задаваме условие за `if`, като не забравяме, че навсякъде използваме ключовата дума `value` и това, с което сме свикнали е да няма празни или бели пространства (като интервали, табулации, нови редове) за тип `string` или числа `< 0`, но поради задачата, се допитах до ИИ за колко е възприета минималната дължина на река. Отговорът е, че няма официално число, но в България, най-късата река е 19 км. Хвърляме изключение за грешка по познатия начин, като този път използваме най-кратката `ArgumentException`, която да съдържа съобщение. Има, разбира се, много видове `Exception`, но ние ще използваме най-краткото засега. Последният интересен момент е, че когато имаме текст, ние можем да преброим неговите символи с `.Length`, но тази дължина ще включва всичко в `string` – букви, цифри, запетайки, интервали, с една дума – всички символи, които съдържа. Ако добавим `.Split()`, то ще преброи думите, т.к. прави абсолютно същото, което и при масиви – разделя. Ако между скобите `()` няма нищо, за разделител ще счита интервал. Ако зададем друг конкретен разделител като `.Split(' ')`, за разделител ще възприеме точка и ще преброи изреченията (но може с неточност, заради съкращенията).

Време да си направим конструктора, като целта е да свържем `public property` заради валидациите, с променливите, които ще подаваме от `Main()`.

```

public River(string name, int length, bool isFloatable, string description)
{
    Name = name;
    Length = length;
    IsFloatable = isFloatable;
    Description = description;
}

```

```

public River(string name, int length, bool isFloatable, string description)
{
    Name = name;
    Length = length;
    IsFloatable = isFloatable;
    Description = description;
}

```

В нашия случай няма нужда от `this.name` – защото ще отиде още по-нависоко и ще хване `private name`, който, обаче няма валидация (защо слагахме `set?`).

ЗАПОМНЕТЕ: Използваме главните букви като защита навсякъде, където искаме да има валидация!!!

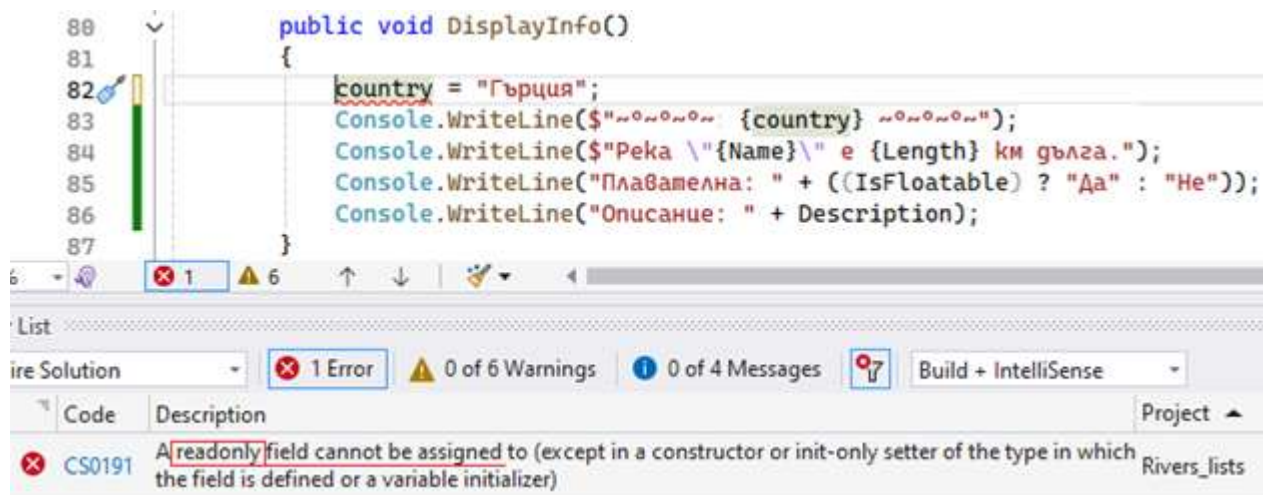
Създаваме си и отпечатването за нашия метод `DisplayInfo()`:

```
public void DisplayInfo()
{
    //country = "Гърция"; забранено, заради readonly
    Console.WriteLine($"~~~~~ {country} ~~~~~");
    Console.WriteLine($"Река \"{Name}\" е {Length} км дълга.");
    Console.WriteLine("Плавателна: " + ((IsFloatable) ? "Да" : "Не"));
    Console.WriteLine("Описание: " + Description);
}
```

В коментар е експеримент да проверим наистина ли `readonly` дава защита, или можем да променим името на държавата -> Гърция. За плавателна е вложена конструкция с тернарния оператор (`condition ? true : false`) – какво да се случи, ако даденото условие е вярно / не. В случая при единия случай ще се отпечата Да, а в другия Не. Тернарният оператор `?:` е подходящ точно за такива кратки условия, вместо `if-else`, защото е негов съкратен вариант `if ? true : else false`.

```
public void DisplayInfo()
{
    //country = "Гърция";
    Console.WriteLine($"~~~~~ {country} ~~~~~");
    Console.WriteLine($"Река \"{Name}\" е {Length} км дълга.");
    Console.WriteLine("Плавателна: " + ((IsFloatable) ? "Да" : "Не"));
    Console.WriteLine("Описание: " + Description);
}
```

И нека направим първия експеримент: да се опитаме да променим името на държавата – вместо България -> Гърция (направо с разкоментиране):



Когато прочетем грешката, тя ни казва, че полето (`private field!`) не може да бъде променено, защото е `readonly`. Може отново да го закоментираме, за да не ни пречи.

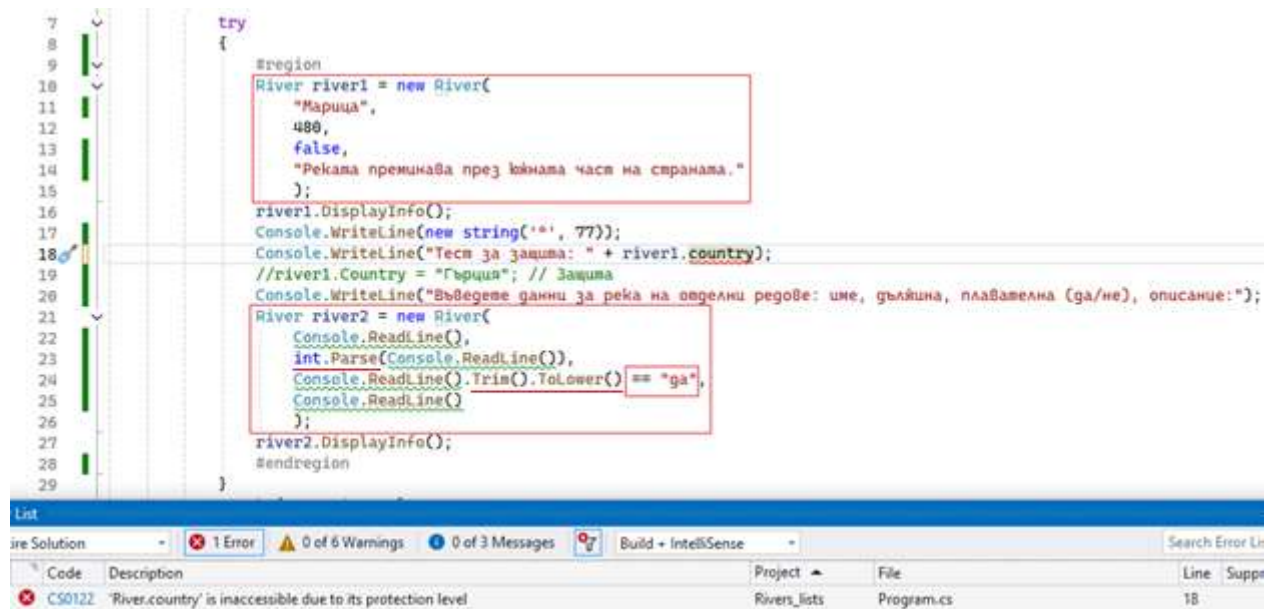
Понеже е private, той е видим само в класа, в който се намира, но ако искаме да го достъпим в Main(), друго решение, за да го направим види, но без да можем да го променяме, е да добавим public property само с get, като напомним, че когато са в комплект { get = read; set = write; }, а когато са поединично { get; } = readonly, което е подходящо само за id и различни постоянни идентификатори като username, а само { set; } = writeonly е само за запис, без възможност за прочитане (като въвеждане на парола).

```
class River
{
    private readonly string country = "България"; // защитен и невидим в Main()
    private string name;
    private int length;
    private string description;

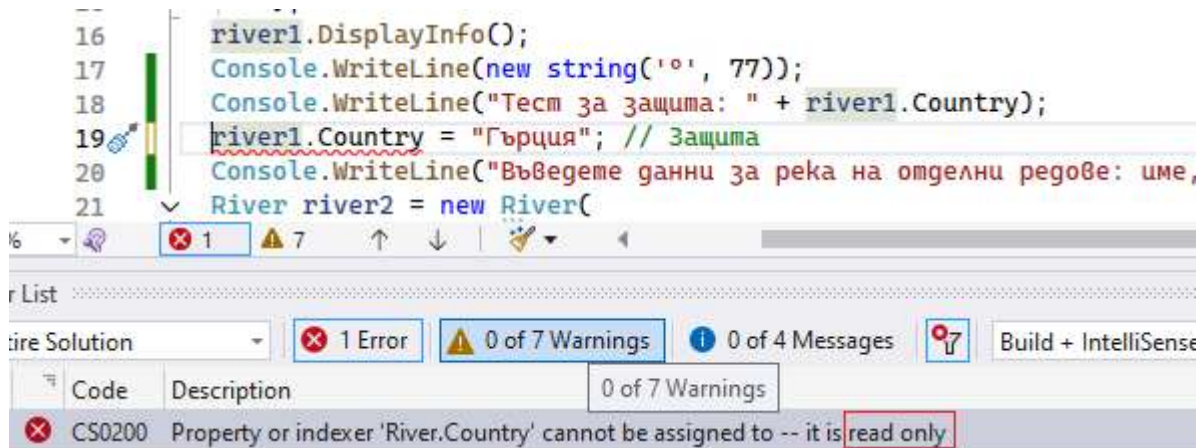
    0 references
    public string Country { get; } // видим в Main()
    2 references
    public string Name
}
```

За всеки private важи правилото, че той е видим само в класа, в който се намира (обхват на видимост между {}). Ако искаме директно да можем да го показваме в Main, му е необходим междинен public. Тази функция изпълняваше public методът DisplayInfo() и чрез него можем да видим скрития и защитен private. Ако искаме солово да можем да показваме country в Main(), го правим public string Country { get; } // видим в Main()

Можем да си създадем нов обект river1 по познатия начин, като единствената допълваща разлика е, че сме подредили входните данни една под друга за по-добра четимост.



Като типичен private нямаме никаква видимост към river1.country и съобщението за грешка ни го изписва – недостъпност поради нивото си на защита.



Другият експеримент е, ако се опитаме да достъпим river1.Country – не хвърля компилационна грешка, но после няма да я покаже на конзолата. И другият тест за грешка е, ако се опитаме да преименуваме държавата от България -> Гърция.

```

D:\Exercises\11 class\Rivers_lists\bin\Debug\net8.0\Rivers_lists.exe
~°~°~°~ България ~°~°~°~
Река "Марица" е 480 км дълга.
Плавателна: Не
Описание: Реката преминава през южната част на страната.
~°~°~°~
Тест за защита:
Въведете данни за река на отделни редове: име, дължина, плавателна (да/не), описание:

```

Въвеждане от CR:

```

Microsoft Visual Studio Debug Console
~°~°~°~ България ~°~°~°~
Река "Марица" е 480 км дълга.
Плавателна: Не
Описание: Реката преминава през южната част на страната.
~°~°~°~
Тест за защита:
Въведете данни за река на отделни редове: име, дължина, плавателна (да/не), описание:
Янтра
286
не
Тя обикаля около старата столица Велико Търново
~°~°~°~ България ~°~°~°~
Река "Янтра" е 286 км дълга.
Плавателна: Не
Описание: Тя обикаля около старата столица Велико Търново
D:\Exercises\11 class\Rivers_lists\bin\Debug\net8.0\Rivers_lists.exe (process 53168) exited with code 0.
Press any key to close this window . . .

```

Нека си тестваме и грешки:

```
Microsoft Visual Studio Debug Console
~^~^~ България ~^~^~
Река "Марица" е 480 км дълга.
Плавателна: Не
Описание: Реката преминава през южната част на страната.
oooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo
Тест за защита:
Въведете данни за река на отделни редове: име, дължина, плавателна (да/не), описание:
Искър
-60
Не
Минава през столицата София
Грешка: Под 19 км е поток!

D:\Exercises\11 class\Rivers_lists\bin\Debug\net8.0\Rivers_lists.exe (process 56248) exited with code 0.
Press any key to close this window . . .
```

Брой думи:

```
Microsoft Visual Studio Debug Console
~^~^~ България ~^~^~
Река "Марица" е 480 км дълга.
Плавателна: Не
Описание: Реката преминава през южната част на страната.
oooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo
Тест за защита:
Въведете данни за река на отделни редове: име, дължина, плавателна (да/не), описание:
Струма
415
не
Стримон
Грешка: Описанието трябва да съдържа поне 3 думи!

D:\Exercises\11 class\Rivers_lists\bin\Debug\net8.0\Rivers_lists.exe (process 54484) exited with code 0.
Press any key to close this window . . .
```


За IT кариерата: могат да се валидират и методи:



```
internal class Program
{
    static void Main(string[] args)
    {
        try
        {
            #region
            River river1 = new River(
                "Марица",
                480,
                false,
                "Реката преминава през южната част на страната."
            );
            river1.DisplayInfo();
            Console.WriteLine(new string('o', 77));
            Console.WriteLine("Тест за защита: " + river1.Country);
            //river1.Country = "Гърция"; // Защита
        }
    }
}
```

```

        Console.WriteLine("Въведете данни за река на отделни редове: име,
дължина, плавателна (да/не), описание:");
        River river2 = new River(
            Console.ReadLine(),
            int.Parse(Console.ReadLine()),
            Console.ReadLine().Trim().ToLower() == "да",
            Console.ReadLine()
        );
        river2.DisplayInfo();
    #endregion
    }
    catch (Exception ex)
    {
        Console.WriteLine("Грешка: " + ex.Message);
    }
    Console.ReadKey();
}

class River
{
    private readonly string country = "България"; // защитен и невидим в Main()
    private string name;
    private int length;
    private string description;

    public string Country { get; } // видим в Main()
    public string Name
    { get => name; // или get { return name; }
      set
      {
          if (string.IsNullOrEmpty(value))
              throw new ArgumentException("Въведете име!");
          name = value;
      }
    }
    public int Length
    { get { return length; }
      set { if (value < 19)
              throw new ArgumentException("Под 19 км е поток!");
            length = value; }
    }
    public string Description
    {
        get => description;
        set
        {
            if (string.IsNullOrEmpty(value) || value.Split().Length < 3) // 2
интервала
                throw new ArgumentException("Описанието трябва да съдържа поне 3
думи!");
            description = value;
        }
    }
    public bool IsFloatable { get; set; }

    public River(string name, int length, bool isFloatable, string description)
    {

```

```
        Name = name;
        Length = length;
        IsFloatable = isFloatable;
        Description = description;
    }

    public void DisplayInfo()
    {
        //country = "Гърция";
        Console.WriteLine($"~~~~~ {country} ~~~~~");
        Console.WriteLine($"Река \"{Name}\" е {Length} км дълга.");
        Console.WriteLine("Плавателна: " + ((IsFloatable) ? "Да" : "Не"));
        Console.WriteLine("Описание: " + Description);
    }
}
```